



Unit testing i VS

Intro



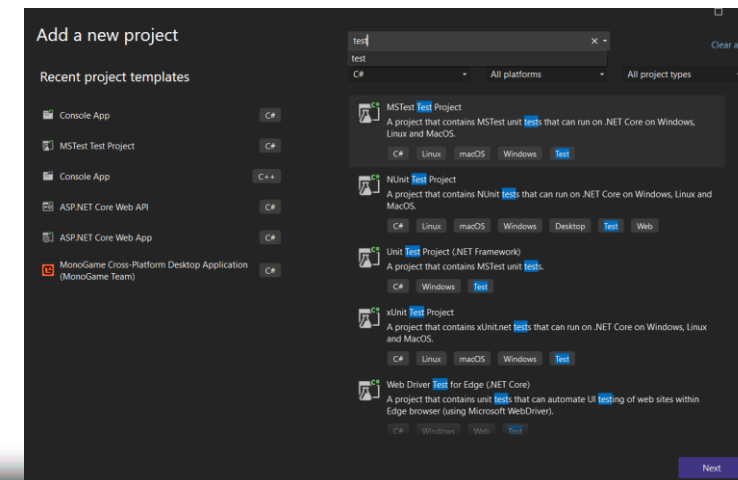
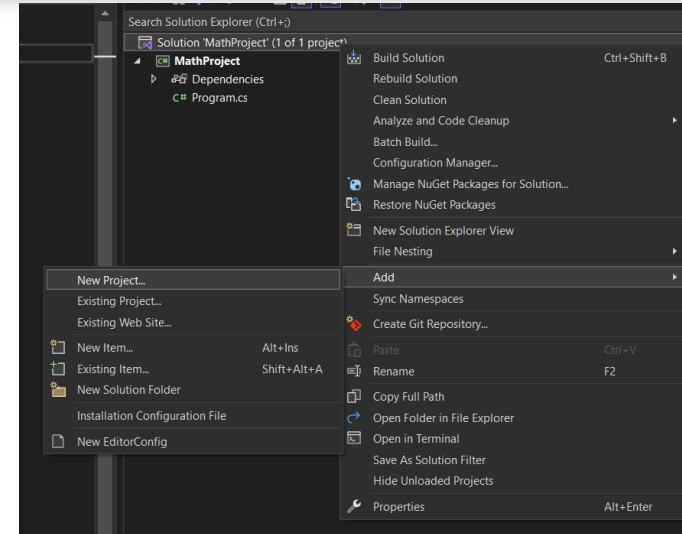
Algoritmer og Unit tests

- Gennem algoritmefaget skal vi arbejde med Unit tests
 - Algoritmer er lette at teste, fordi vi ofte ved hvad vi forventer at få ud af dem.
 - Vi giver dem data og får noget vi kan forvente tilbage.
 - Derfor er det et godt fag at lære Unit testing



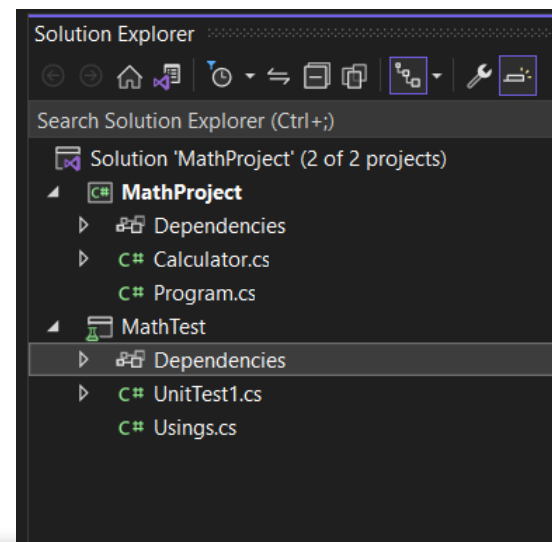
Test projekt

- For at man kan teste, skal man oprette et test project.
- Et test projekt bruges til at køre Unit tests på vores C# kode.
 - For at vi kan teste noget, skal vi derfor også oprette Console projekt
 - Der er flere forskellige test libraries
 - Brug blot MSTest test project



Test projekt

- I følgende eksempler tester vi et simpelt Console projekt, som håndterer matematiske beregninger.
 - Derfor har jeg lavet følgende opsætning i min solution:
 - MathTest: Et test projekt
 - MathProject: Et console projekt
 - Inkluderer en Calculator klasse som vi vil udføre tests på



Unit testing

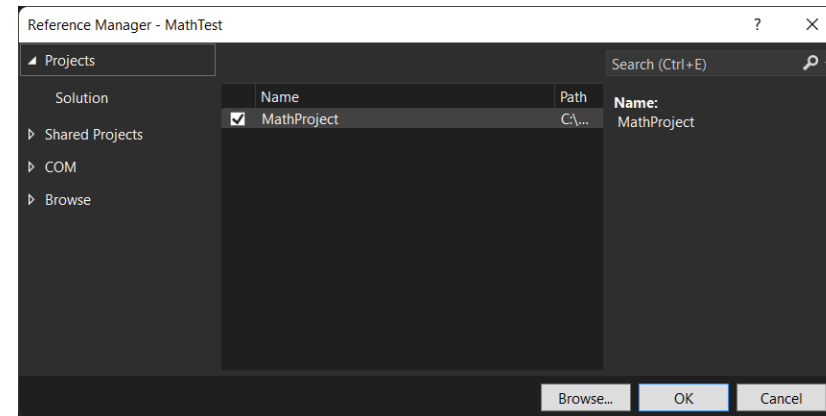
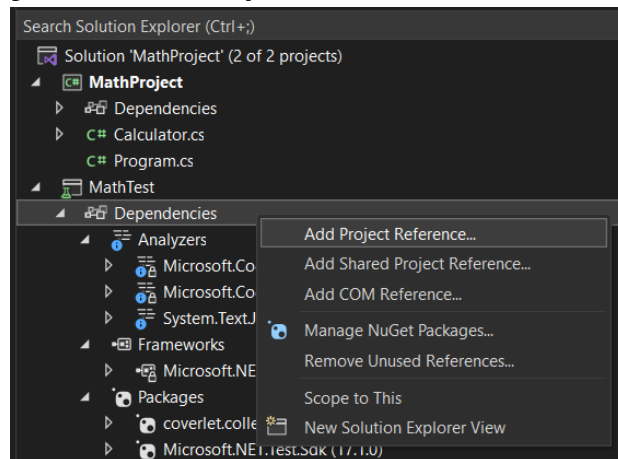
- `[TestMethod]` tag, bruges til at fortælle Visual Studio at denne metode skal testes
- `[TestClass]` tag som bruges til at fortælle at klassen skal testes
- Hvis disse tags fjernes bliver der ikke kørt tests på klassen eller metoden

```
[TestClass]
0 references
public class UnitTest1
{
    [TestMethod]
    0 references
    public void TestMethod1()
    {
    }
}
```



Reference

- For at man kan teste et projekt, skal der først laves en reference til det
 - Dvs. Test projektet skal have en reference til dit console projekt
 - Test Project ? Dependencies ? Add Project Reference



Reference

- Det er også vigtigt at vi inkludere namespace fra det projekt som holder på den funktionalitet vi vil teste
 - Dvs. Vi bør inkludere følgende namespace i vores test projekt:

```
using System;  
using Microsoft.Visu  
using MathProject;
```

- Hvis vi glemmer dette, så er det ikke muligt for os at teste klasserne i projektet, og derved kan vi ikke skrive unit tests til Calculator klassen



Klasser

- **VIGTIGT!**
 - Jeres klasser skal være public for at I kan få fat på dem i jeres test projekt
 - **Note:** Hvis en klasse ikke er public, skal I explicit gøre den synlig for testprojektet ved at bruge: `[assembly: InternalsVisibleTo("MathTestInternalClass")]`

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Runtime.CompilerServices;
5 using System.Text;
6 using System.Threading.Tasks;
7 [assembly: InternalsVisibleTo("MathTestInternalClass")]
8
9 namespace MathProjectInternalClassTest
10 {
11     5 references
12     internal class Calculator
13     {
14         /// <summary>
15         /// Adds two integers
16         /// </summary>
17         /// <param name="first">The first value</param>
18         /// <param name="second">The second value</param>
19         /// <returns></returns>
20         1 reference
21         internal int Add(int first, int second)
22         {
23         }
24     }
25 }
```

Search Solution Explorer (Ctrl+)

- Solution 'MathProjectInternalClassTest' (2 of 2 projects)
- MathProjectInternalClassTest
 - Properties
 - References
 - App.config
 - C# Calculator.cs
 - C# Program.cs
 - MathTestInternalClass**
 - Dependencies
 - C# GlobalUsings.cs
 - C# UnitTest1.cs

82 % No issues found Ln: 11 Ch: 30 SPC CRLF

Klasser

- I dette eksempel laver vi en Calculator klasse

```
public class Calculator
```



Arrange ? Act ? Assert

- Når vi laver unit test bruger man en fremgangsmåde, som hedder Arrange, Act, Assert
- Arrange:
 - Opretter objekter og variabler, som skal indgå i testen
- Act:
 - Kører den funktionalitet, som skal testes
- Assert:
 - Tester om resultatet af Act er det vi forventer.



Add

- Vi kan oprette en Add metode i vores Calculator, som ser ud på følgende måde:

```
/// <summary>
/// Adds two integers
/// </summary>
/// <param name="first">The first value</param>
/// <param name="second">The second value</param>
/// <returns></returns>
public int Add(int first, int second)
{
    return first + second;
}
```



Arrange ? Act ? Assert

- Eksempel på Test af Add
 - Assert er det kode, som køres af Test frameworket
 - Hvis vi glemmer Assert, så bliver testen ikke kørt korrekt(får falsk positiv).

```
[TestMethod]
0 references
public void TestAdd()
{
    //Arrange
    Calculator calculator = new Calculator();
    int first = 10;
    int second = 15;
    int expected = 25;

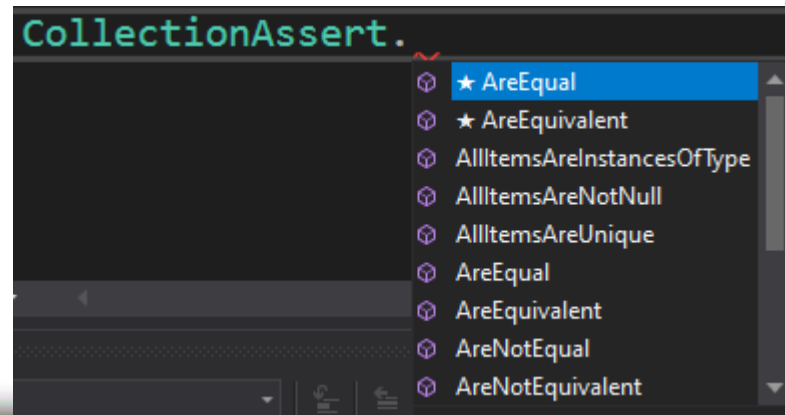
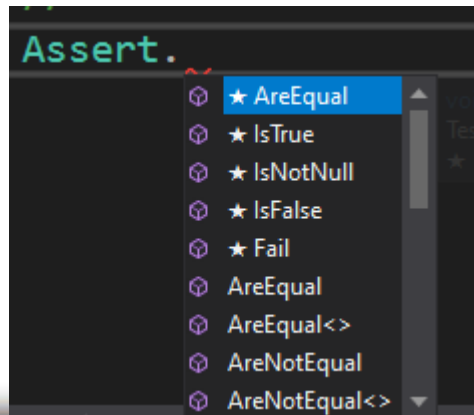
    //Act
    int actual = calculator.Add(first, second);

    //Assert
    Assert.AreEqual(expected, actual);
}
```



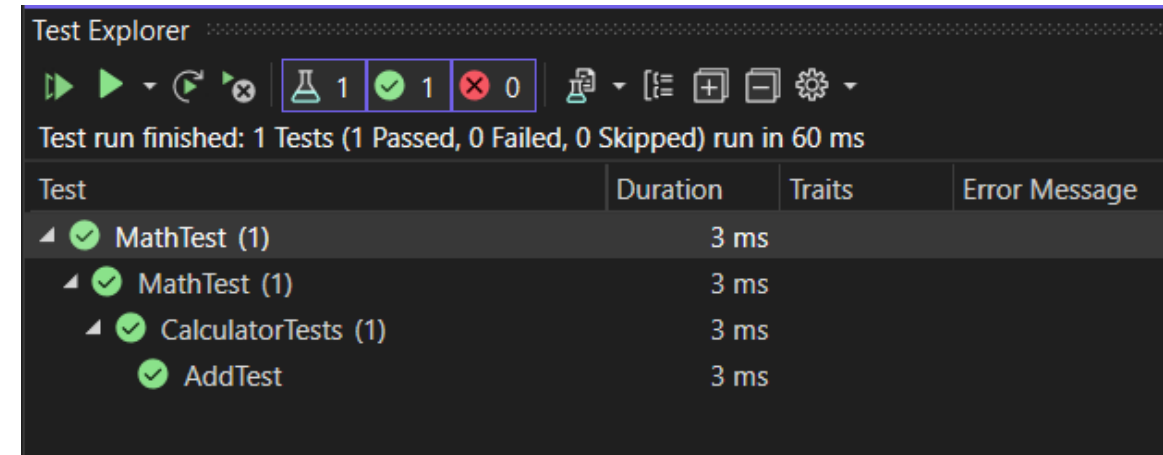
Assert

- Der findes mange forskellige typer af Asserts.
 - Hvis i skriver Assert. Kan i se de muligheder i har.
 - Der findes også en CollectionAssert, som bruges til at sammenligne Collections (vi ser nærmere på den senere)
 - **Man bør kun have en assert per test.**

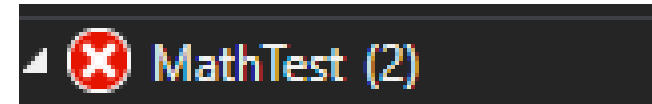


Kørsel af tests

- Test Explorer → Run all tests:



- Kan også køres direkte fra vinduet "Test Explorer"
- Hvis testen går godt får man et grønt flueben.
- Hvis den fejler får man et rødt kryds:



Øvelse

1. Opret et test projekt
2. Opret et console projekt
3. Opret en Calculator klasse i jeres console projekt
4. Opsæt en reference til Console projektet i testprojektet
5. Opret en instans af Calculator klassen i test metoden

```
namespace MathTest
{
    [TestClass]
    0 references
    public class CalculatorTest
    {
        [TestMethod]
        0 references
        public void TestInstantiate()
        {
            Calculator calculator = new Calculator();
        }
    }
}
```



Øvelse

- Implementer en Add metode og lav en unit test af den.



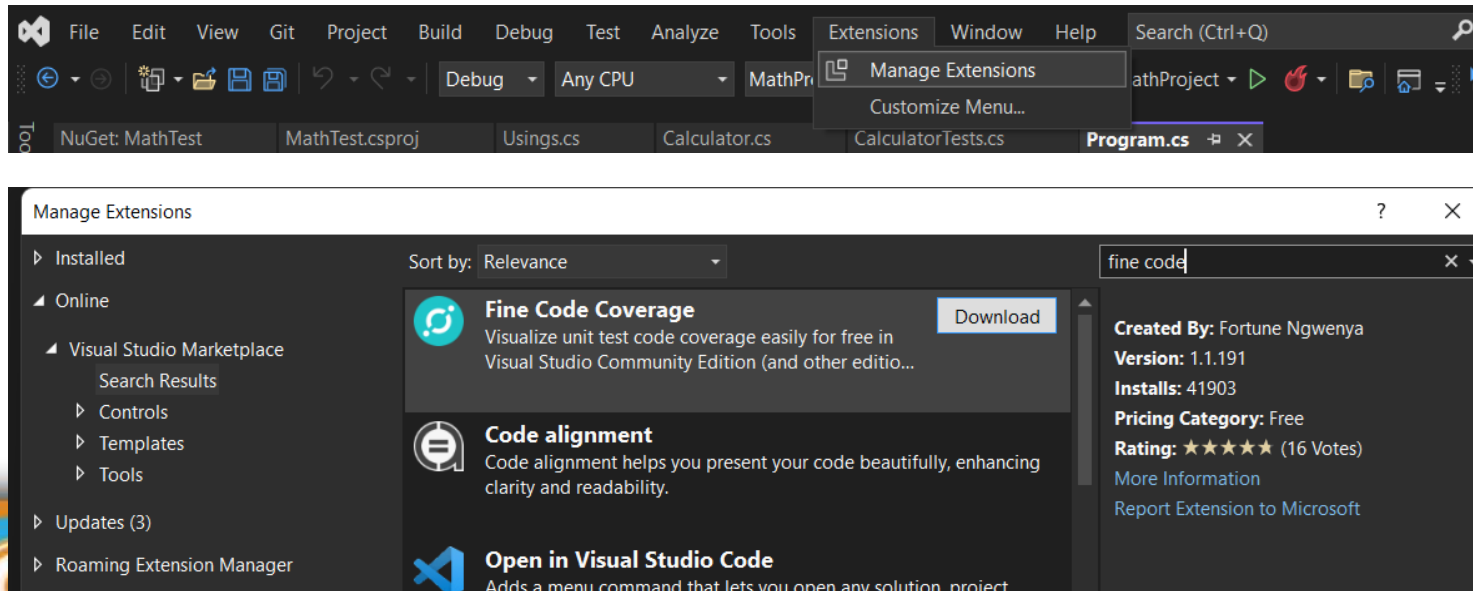
Code coverage

- Her er en liste af de kriterier, som definerer om vi har code coverage
 - Method coverage:
 - Er alle metoder i programmet blevet kaldt?
 - Condition coverage:
 - Er alle bolske udtryk blevet evalueret som true og false
 - Branch coverage
 - Er alle branches blevet kørt. Dvs. if sætninger og switch cases. F.eks. Er både true og false branch blevet kørt i alle if sætninger. Else sætninger indgår også her
 - Statement coverage:
 - Er alle statements blevet eksekveret. Alle linjer kode



Code coverage

- Fine Code coverage - findes [Her](#)
 - En extension der giver mulighed for at få visualiseret sin code coverage
 - Kan også hentes fra extensions->manage extensions->visual studio marketplace
 - OBS: VS skal genstartes for at installere.



Code coverage

- Method Coverage 1/4
 - Vi har en calculator klasse med følgende metoder:

```
public bool NumberIsEven(int number)
{
    if (number % 2 == 0)
    {
        Console.WriteLine("The number is even");
        return true;
    }
    else
    {
        Console.WriteLine("The number is odd");
        return false;
    }
}
```

```
public bool NumberIsPositive(int number)
{
    if (number > 0)
    {
        Console.WriteLine("Number is positive");
        return true;
    }
    else if (number < 0)
    {
        Console.WriteLine("Number is negative");
        return false;
    }
    else
    {
        Console.WriteLine("Number is zero");
        return false;
    }
}
```



Code coverage

- Method Coverage 2/4
 - For at få 100% method coverage skal vi lave følgende tests

```
[TestMethod]
public void NumberIsPositive()
{
    //Arrange
    Calculator calculator = new Calculator();
    int number = 10;
    bool expected = true;

    //Act
    bool actual = calculator.NumberIsPositive(number);

    //Assert
    Assert.AreEqual(expected, actual);
}
```

```
public bool NumberIsPositive(int number)
{
    if (number > 0)
    {
        Console.WriteLine("Number is positive");
        return true;
    }
    else if (number < 0)
    {
        Console.WriteLine("Number is negative");
        return false;
    }
    else
    {
        Console.WriteLine("Number is zero");
        return false;
    }
}
```



Code coverage

- Method Coverage 3/4
 - For at få 100% method coverage skal vi lave følgende tests

```
[TestMethod]
public void NumberIsEven()
{
    //Arrange
    Calculator calculator = new Calculator();
    int number = 10;
    bool expected = true;

    //Act
    bool actual = calculator.NumberIsEven(number);

    //Assert
    Assert.AreEqual(expected, actual);
}
```

```
public bool NumberIsEven(int number)
{
    if (number % 2 == 0)
    {
        Console.WriteLine("The number is even");
        return true;
    }
    else
    {
        Console.WriteLine("The number is odd");
        return false;
    }
}
```



Code coverage

- Method Coverage 4/4
 - Vi kan endda se på vores metoder at de bliver kaldt!

```
1 reference | 1/1 passing
public bool NumberIsEven(int number)
{
    if (number % 2 == 0)
    {
        Console.WriteLine("The number is even");
        return true;
    }
    else
    {
        Console.WriteLine("The number is odd");
        return false;
    }
}
```

```
1 reference | 1/1 passing
public bool NumberIsPositive(int number)
{
    if (number > 0)
    {
        Console.WriteLine("Number is positive");
        return true;
    }
    else if (number < 0)
    {
        Console.WriteLine("Number is negative");
        return false;
    }
    else
    {
        Console.WriteLine("Number is zero");
        return false;
    }
}
```



Code coverage

- Kigger vi i vinduet for Fine Code coverage nu kan vi se følgende:

Collapse all | Expand all Filter:

▼ Name	▼ Covered	▼ Uncovered	▼ Coverable	▼ Total	▼ Line coverage		▼ Covered	▼ Total	▼ Branch coverage	
– MathsProject	12	12	24	60	50%		2	6	33.3%	
MathsProject.Calculator	12	10	22	45	54.5%		2	6	33.3%	
MathsProject.Program	0	2	2	15	0%		0	0		
+ MathsProjectUnitTest	14	0	14	55	100%		0	0		

- Vi kan se at vi har 54.5% line coverage og kun 33.3% (2/6) branch coverage.
 - Den påstår at der er 6 cases, jeg kan dog kun se 5 i Calculator klassen 😊
 - Der skal bruges flere tests



Code coverage

- Branch coverage 1/3
 - For at få 100% condition coverage skal følgende tests køres.

```
[TestMethod]
public void NumberIsEvenFalse()
{
    //Arrange
    Calculator calculator = new Calculator();
    int number = 9;
    bool expected = false;

    //Act
    bool actual = calculator.NumberIsEven(number);

    //Assert
    Assert.AreEqual(expected, actual);
}
```

```
public bool NumberIsEven(int number)
{
    if (number % 2 == 0)
    {
        Console.WriteLine("The number is even");
        return true;
    }
    else
    {
        Console.WriteLine("The number is odd");
        return false;
    }
}
```

- 3/6



Code coverage

- Branch coverage 2/3
 - For at få 100% condition coverage skal følgende tests køres.

```
[TestMethod]
public void NumberIsPositiveFalse()
{
    //Arrange
    Calculator calculator = new Calculator();
    int number = -10;
    bool expected = false;

    //Act
    bool actual = calculator.NumberIsPositive(number);

    //Assert
    Assert.AreEqual(expected, actual);
}
```

```
public bool NumberIsPositive(int number)
{
    if (number > 0)
    {
        Console.WriteLine("Number is positive");
        return true;
    }
    else if (number < 0)
    {
        Console.WriteLine("Number is negative");
        return false;
    }
    else
    {
        Console.WriteLine("Number is zero");
        return false;
    }
}
```

- 5/6 – denne giver os også den til den hemmelige case



Code coverage

- Branch coverage 3/3
 - For at få 100% condition coverage skal følgende tests køres.

```
[TestMethod]
public void NumberIsZeroFalse()
{
    //Arrange
    Calculator calculator = new Calculator();
    int number = 0;
    bool expected = false;

    //Act
    bool actual = calculator.NumberIsPositive(number);

    //Assert
    Assert.AreEqual(expected, actual);
}
```

- 6/6

```
public bool NumberIsPositive(int number)
{
    if (number > 0)
    {
        Console.WriteLine("Number is positive");
        return true;
    }
    else if (number < 0)
    {
        Console.WriteLine("Number is negative");
        return false;
    }
    else
    {
        Console.WriteLine("Number is zero");
        return false;
    }
}
```



Code coverage

- Kigger vi i vinduet for Fine Code coverage nu kan vi se følgende:

Collapse all | Expand all Filter:

▼ Name	▼ Covered	▼ Uncovered	▼ Coverable	▼ Total	▼ Line coverage	▲ Covered	▼ Total	▼ Branch coverage
– MathsProject	22	2	24	58	91.6%	6	6	100%
MathsProject.Calculator	22	0	22	43	100%	6	6	100%
MathsProject.Program	0	2	2	15	0%	0	0	
+ MathsProjectUnitTest	35	0	35	83	100%	0	0	

- Vi har 100% 😊
- Disse 100% betyder vi har 100% statement coverage.



Code coverage

- Mht Condition coverage, så hænger denne tæt sammen med vores branch coverage i denne case.
- Vi kunne godt have tilføjet flere test for at gøre det endnu mere explicit.
- Hellere én test for meget end én for lidt!



Øvelse

- Implementer en metode som returnerer prisen på en Togbillet baseret på alder:
 - 0 til 1 år: 0 kr.
 - 2 til 14 år: 10 kr.
 - 15-64 år: 20 kr.
 - +65: 5 kr.
- Test metoden og sørg for at I har 100% code coverage



Ækvivalensklasser aka Equivalence Classes

- Det kan være svært at vide, hvor mange værdier man skal teste, og nogle tilfælde kræver mange tests.
- For at reducere antallet af tests kan input opdeles i ækvivalensklasser (kategorier), hvor én test fra hver klasse repræsenterer hele gruppen.

**Equivalence Class
Partitioning**



Ækvivalensklasser aka Equivalence Classes

- Hvis en metode tager et tal mellem 1 og 12 og returnerer en måned, kan vi opdele input i ækvivalensklasser:
 - Ugyldige værdier: < 1 (f.eks. -2)
 - Gyldige værdier: $1 - 12$ (f.eks. 6)
 - Ugyldige værdier: > 12 (f.eks. 15)
- Ved at teste én værdi fra hver klasse (-2, 6, 15) kan vi validere metoden effektivt uden at teste alle tal.



Grænseværdier

Ækvivalensklasser alene fanger ikke "off-by-one errors".

For at sikre robuste tests bør vi kombinere dem med grænseværdier:

- Lav grænse: 0, 1
- Høj grænse: 12, 13

Dette tester overgangene mellem gyldige og ugyldige værdier, hvor fejl ofte opstår.



Collection Assert

- Når man vil køre Assert på collections skal man gøre brug af CollectionAssert.
- Hvis man ikke gør brug af den kan man få fejl i sine tests.
 - Normal assert sammenligner objekter
 - Collection assert sammenligner indholdet af en collection.



Collection Assert

- Eksempel

- Vi har en liste af alle de resultater, som lommeregneren har beregnet

```
public List<int> Results { get; set; } = new List<int>();
```

- I Add metoden tilføjes følgende

```
public int Add(int first, int second)
{
    Results.Add(first + second);
    return first + second;
}
```



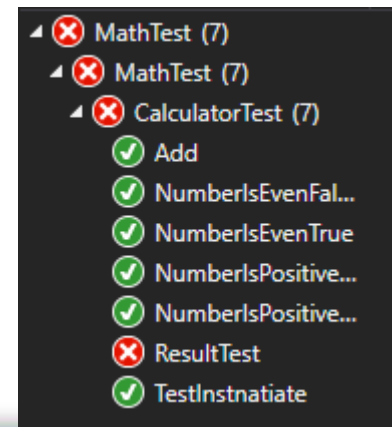
Collection Assert

- Hvis vi tester med Assert får vi følgende resultat
 - Med Assert sammenlignes objekterne. Expected er ikke den samme liste som Calculator.results i hukommelsen
 - Derfor fejler testen.

```
public void ResultTest()
{
    //Arrange
    Calculator calculator = new Calculator();
    List<int> expected = new List<int>(new int[] { 16, 10 });

    //Act
    calculator.Add(13, 3);
    calculator.Add(9, 1);

    //Assert
    Assert.AreEqual(expected, calculator.Results);
}
```



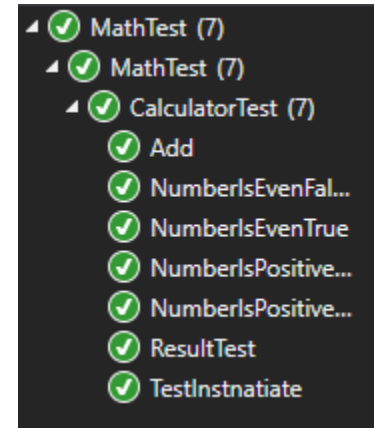
Collection Assert

- Med CollectionAssert sammenlignes indholdet af listerne.
 - Derfor bliver testen positiv.

```
public void ResultTest()
{
    //Arrange
    Calculator calculator = new Calculator();
    List<int> expected = new List<int>(new int[] { 16, 10 });

    //Act
    calculator.Add(13, 3);
    calculator.Add(9, 1);

    //Assert
    CollectionAssert.AreEqual(expected, calculator.Results);
}
```



Exceptions

- I nogle tilfælde kan det være tiltænkt at en metode skal kaste en exception. Det kan håndteres på flere måder
 - Man kan anvende et ExpectedException tag

```
public void NumberIsTen(int input)
{
    if (input == 10)
    {
        Console.WriteLine("The number is 10");
    }
    else
    {
        throw new NumberNotTenException("The number is not ten");
    }
}
```



Exceptions

- Kan testes på følgende måde

```
[TestMethod]
[ExpectedException(typeof(NumberNotTenException))]
public void NumberIsTenFalse()
{
    //Arrange
    Calculator calculator = new Calculator();

    //Act
    calculator.NumberIsTen(3);
}
```



Exceptions

- Man kan også anvende en specifik Assert:

```
[TestMethod]
public void NumberIsTenFalse()
{
    //Arrange
    Calculator calculator = new Calculator();

    //Act & Assert
    Assert.ThrowsException<NumberNotTenException>(() => calculator.NumberIsTen(3));
}
```



Øvelse

- Lav en metode som fjerner alle ulige tal fra en liste.
 - Brug collectionasset til at unit test metoden.
 - Feks. Hvis metoden får 1,2,3,4,5,6,7,8,9,10
 - Skal den returnere en liste med: 2,4,6,8,10



Øvelse

- Lav en metode, som returnerer navnet på en måned baseret på en int.
- Brug ækvivalensklasser i sammenhold med grænseværdier til at lave fyldestgørende unit tests.



Øvelse

- Største tal i en mængde
 - Lav en algoritme/metode, som finder det største tal i et array.
 - Hvis arrayet er tomt skal der kastes en exception.
 - Du bestemmer selv hvilken exception du kaster, du må godt lave din egen exception type.
 - Lav fyldestgørende Unit tests på algoritmen

